

2025-2026 BAHAR DÖNEMİ
BİLGİSAYAR MÜHENDİSLİĞİ SİSTEM PROGRAMLAMA

PROJE ÖDEVİ RAPORU

Ders: Sistem Programlama

Dönem: 2025-2026 Bahar

Teslim: 24 Mayıs 2026

Grup Üyeleri:

- Ömer Akkıyal - G231210058
- Ramazan Katran - G231210076

GitHub Adresi:

<https://github.com/omer-akkiyal/sistem-programlama-proje-odevi>

1. PROJE TANIMI VE AMAÇ

Bu projede, tar, rar veya zip benzeri çalışan ancak sıkıştırma yapmayan "tarsau" adlı bir arşivleme programı geliştirilmiştir. Program Linux işletim sisteminde C dilinde yazılmış olup komut satırından çalıştırılmaktadır. Program iki temel işlevi desteklemektedir:

- **tarsau -b** : Birden fazla metin dosyasını tek bir .sau arşiv dosyasında birleştirme
- **tarsau -a** : Bir .sau arşiv dosyasını belirtilen dizine veya mevcut dizine çıkartma

2. GELİŞTİRME SÜRECİ

Proje geliştirme süreci aşağıdaki aşamalardan oluşmaktadır:

1. **Aşama 1 - Tasarım:** Öncelikle .sau arşiv dosyasının formatı belirlenmiştir. Format üç bölümden oluşmaktadır: 10 baytlık header, organizasyon bölümü ve dosya verileri. Bu tasarım, arşiv dosyasının parse edilebilir ve genişletilebilir olmasını sağlamaktadır.
2. **Aşama 2 - Modüler Yapı:** Proje üç dosyaya bölünmüştür:
 - **tarsau.h** : Sabitler ve fonksiyon prototipleri
 - **tarsau.c** : Temel işlevler (arşivleme, çıkartma, yardımcı fonksiyonlar)
 - **main.c** : Komut satırı ayrıştırması ve işlem yönlendirmesi
3. **Aşama 3 - Temel Fonksiyonlar:** `is_ascii()`, `get_file_size()`, `get_file_permissions()` yardımcı fonksiyonları yazılmıştır. Ardından `archive_files()` ve `extract_archive()` ana fonksiyonları gerçekleştirilmiştir.
4. **Aşama 4 - Hata Yönetimi:** ASCII olmayan dosya reddi, dosya boyut sınırı (200 MB), dosya sayısı sınırı (32), bozuk arşiv algılama gibi hata durumları eklenmiştir.
5. **Aşama 5 - Test:** Farklı senaryolarla (tek dosya, çok dosya, hatalı dosya, izin açma) testler yapılarak doğruluk sağlanmıştır.

3. SAU ARŞİV DOSYASI FORMATI

Arşiv dosyası iki bölümden oluşmaktadır:

1) Organizasyon (İçerik) Bilgileri:

- İlk 10 bayt, organizasyon bölümünün ASCII formatındaki sayısal boyutunu içerir (sıfır dolgulu, örneğin: "0000000043").
- Sonraki her kayıt '|' karakteri ile ayrılır.
- Kayıttaki alanlar virgül ile ayrılmıştır: |Dosya adı, izinler (octal), boyut (bayt)|
- Örnek: |t1.txt,644,29|t2.txt,644,20|t3.txt,644,21|

2) Arşivlenmiş Dosyalar:

- Arşivlenmiş dosyalar, herhangi bir ayırıcı kullanılmadan ASCII formatında art arda yerleştirilir.
- Her dosyanın sınırı, organizasyon bölümündeki boyut bilgisiyle belirlenir.

Örnek arşiv içeriği (s1.sau):

```
0000000043|t1.txt,644,29|t2.txt,644,20|t3.txt,644,21|Bu birinci dosyanın içeriği.  
Bu da ikinci dosya.  
Üçüncü dosya burada.
```

4. KAYNAK KOD VE AÇIKLAMALARI

4.1. tarsau.h - Başlık Dosyası

```
#ifndef  
TARSAU_H  
#define  
TARSAU_H
```

```

#define MAX_FILES 32 // Giriş dosyası sayısı en fazla
32
#define MAX_TOTAL_SIZE (200 * 1024 * 1024) // Toplam boyut 200 MB'ı
geçemez
int is_ascii(const char *filename);
long get_file_size(const char *filename);
int get_file_permissions(const char *filename);
void archive_files(int file_count, char *files[], const char *output_file);
void extract_archive(const char *archive_file, const char *target_dir);

#endif

```

Açıklama: MAX_FILES ve MAX_TOTAL_SIZE sabitleri, proje gereksinimlerinde belirtilen sınırları tanımlar. Fonksiyon prototipleri tarsau.c'deki fonksiyonların main.c tarafından kullanılabilmesini sağlar.

4.2. main.c - Ana Program

```

#include <stdio.h>
#include <string.h>
#include "tarsau.h"

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Kullanım: tarsau -b dosya1 dosya2 ... [-o cikti.sau] VEYA
tarsau -a arshiv.sau [dizin]\n");
        return 1;
    }

    if (strcmp(argv[1], "-b") == 0) {
        char *input_files[MAX_FILES];
        int file_count = 0;
        char *output_file = "a.sau"; // Varsayılan çıktı dosyası
        long total_size = 0;

        // Komut satırı argümanlarını ayrıştırma
        for (int i = 2; i < argc; i++) {
            if (strcmp(argv[i], "-o") == 0) {
                if (i + 1 < argc) {
                    output_file = argv[i + 1];
                    break;
                }
            }
        }
    }
}

```

```

        }
    } else {
        if (file_count >= MAX_FILES) {
            printf("Hata: Giriş dosyası sayısı en fazla 32
olabilir.\n");
            return 1;
        }
        input_files[file_count++] = argv[i];
    }
}

// Giriş dosyalarının uygunluğunu denetleme
for (int i = 0; i < file_count; i++) {
    if (!is_ascii(input_files[i])) {
        printf("%s giriş dosyasının formatı uyumsuzdur!\n",
input_files[i]);
        return 1;
    }
    long size = get_file_size(input_files[i]);
    if (size == -1) {
        printf("Hata: %s dosyası okunamadı!\n", input_files[i]);
        return 1;
    }
    total_size += size;
}

if (total_size > MAX_TOTAL_SIZE) {
    printf("Hata: Giriş dosyalarının toplam boyutu 200 MB'ı
geçemez.\n");
    return 1;
}

archive_files(file_count, input_files, output_file);
}
else if (strcmp(argv[1], "-a") == 0) {
    if (argc < 3 || argc > 4) {
        printf("Kullanım: tarsau -a arxiv.sau [dizin]\n");
        return 1;
    }
    char *archive_file = argv[2];
    char *target_dir = NULL;
    if (argc == 4) {
        target_dir = argv[3];
    }
}

```

```

    }
    extract_archive(archive_file, target_dir);
}
else {
    printf("Geçersiz parametre.\n");
}

return 0;
}

```

Açıklama: main() fonksiyonu komut satırı argümanlarını ayrıştırır. "-b" modunda giriş dosyalarının ASCII uyumluluğu, dosya sayısı (max 32) ve toplam boyut (max 200 MB) kontrol edildikten sonra arşivleme fonksiyonu çağrılır. "-a" modunda arşiv dosyası ve isteğe bağlı hedef dizin alınarak çıkartma fonksiyonu çağrılır. Varsayılan arşiv dosya adı "a.sau"dur.

4.3. tarsau.c - Temel İşlevler

is_ascii() Fonksiyonu

```

int is_ascii(const char *filename) {
    FILE *f = fopen(filename, "rb");
    if (!f) return 0;
    int c;
    while ((c = fgetc(f)) != EOF) {
        if (c < 0 || c > 127) {
            fclose(f);
            return 0;
        }
    }
    fclose(f);
    return 1;
}

```

get_file_size() Fonksiyonu

```

long get_file_size(const char *filename) {
    struct stat st;
    if (stat(filename, &st) == 0) {

```

```

        return st.st_size;
    }
    return -1;
}

```

get_file_permissions() Fonksiyonu

```

int get_file_permissions(const char *filename) {
    struct stat st;
    if (stat(filename, &st) == 0) {
        return st.st_mode & 0777;
    }
    return 0;
}

```

archive_files() Fonksiyonu

```

void archive_files(int file_count, char *files[], const char *output_file)
{
    FILE *out = fopen(output_file, "w");
    if (!out) {
        printf("Hata: Çıktı dosyası (%s) oluşturulamadı!\n", output_file);
        return;
    }

    char org_buffer[8192] = "";
    for (int i = 0; i < file_count; i++) {
        long size = get_file_size(files[i]);
        int perms = get_file_permissions(files[i]);
        char temp_info[256];
        sprintf(temp_info, "|%s,%o,%ld", files[i], perms, size);
        strcat(org_buffer, temp_info);
    }
    strcat(org_buffer, "|");
    long org_size = strlen(org_buffer);
    fprintf(out, "%010ld", org_size);
    fprintf(out, "%s", org_buffer);

    for (int i = 0; i < file_count; i++) {
        FILE *in = fopen(files[i], "r");
    }
}

```

```

        if (in) {
            int c;
            while((c = fgetc(in)) != EOF) {
                fputc(c, out);
            }
            fclose(in);
        }

    }

    fclose(out);
    printf("Dosyalar birleştirildi.\n");
}

```

extract_archive() Fonksiyonu

```

void extract_archive(const char *archive_file, const char *target_dir) {
    FILE *in = fopen(archive_file, "r");
    if (!in) {
        printf("Arşiv dosyası uygunsuz veya bozuk!\n");
        return;
    }

    char header[11] = {0};
    if (fread(header, 1, 10, in) != 10) {
        printf("Arşiv dosyası uygunsuz veya bozuk!\n");
        fclose(in);
        return;
    }

    long org_size = atol(header);
    if (org_size <= 0) { /* hata mesajı */ }

    char *org_data = malloc(org_size + 1);
    fread(org_data, 1, org_size, in);
    org_data[org_size] = '\0';

    if (target_dir != NULL) {
        struct stat st = {0};
        if (stat(target_dir, &st) == -1) {
            mkdir(target_dir, 0755);
        }
    }
}

```



```

long current_file_pos = 10 + org_size;
char *token = strtok(org_data, "|");

while (token != NULL) {
    char filename[256];
    int perms;
    long size;
    sscanf(token,"%255[^\n],%o,%ld",filename, &perms, &size);

    char out_path[512];
    if (target_dir != NULL)
        sprintf(out_path,"%s/%s",target_dir, filename);
    else
        strcpy(out_path, filename);

    fseek(in, current_file_pos, SEEK_SET);
    FILE *out = fopen(out_path, "w");
    if (out) {
        long bytes_left = size;
        int c;
        while(bytes_left>0&&(c=fgetc(in)) != EOF) {
            fputc(c, out);
            bytes_left--;
        }
        fclose(out);
        chmod(out_path, perms);
    }
    current_file_pos += size;
    token = strtok(NULL, "|");
}

free(org_data);
fclose(in);
}

```

Açıklama: Organizasyon boyutu elde edilir, veri okunur ve '|' ayracıyla parçalanır. Hedef dizin yoksa mkdir() ile oluşturulur. Her kayıttan dosya adı, izin ve boyut bilgisi ayrıştırılır. fseek() ile dosya verisinin başladığı konuma gidilir. Her dosyanın içeriği boyut kadar okunarak yeni dosyaya yazılır. chmod() ile dosyanın orijinal izinleri geri yüklenir.

4.4. Makefile

```
all: tarsau

tarsau: main.c tarsau.c
    gcc -o tarsau main.c tarsau.c -I.

clean:
    rm -f tarsau
```

Açıklama: make komutu ile proje derlenir. gcc, main.c ve tarsau.c dosyalarını birlikte derleyerek "tarsau" çalıştırılabilir dosyasını oluşturur. -I. bayrağı, tarsau.h başlık dosyasının bulunduğu dizini belirtir. make clean komutu derleme sonucu oluşan dosyaları temizler.

5. ÇALIŞMA ÇIKTILARI VE TEST SONUÇLARI

Aşağıda proje testleri sırasında elde edilen örnek terminal ekran görüntüleri ve çıktıları yer almak tadır:

Test 1: Dosyaları Arşivleme ve Make İşlemi

```
resmi@DESKTOP-4RDPMO8:/mnt/c/Users/resmi/Desktop/sistrem programlama/Sistem Proje Ödevi$ make clean
make
./tarsau -b t1.txt t2.txt t3.txt -o test_arsiv.sau
ls -la test_arsiv.sau
rm -f tarsau
gcc -o tarsau main.c tarsau.c -I.
Dosyalar birleştirildi.
-rwxrwxrwx 1 root root 123 May 24 18:32 test_arsiv.sau
```

```
$ ./tarsau -b t1.txt t2.txt t3.txt -o s1.sau
Dosyalar birleştirildi.
$ cat s1.sau
0000000043|t1.txt,644,29|t2.txt,644,20|t3.txt,644,21|Bu birinci dosyanın içeriği.
Bu da ikinci dosya.
Üçüncü dosya burada.
```

Doğrulama:

- Header organizasyon bölümünü başarıyla yazar: "0000000043" -> Organizasyon bölümü 43 bayt. **[DOĞRU]**
- Organizasyon: 3 dosya kaydı, her biri dosya adı, izin ve boyut içeriyor.
- Veri bölümü: $29 + 20 + 21 = 70$ bayt dosya içeriği art arda sorunsuz eklenir. **[DOĞRU]**
- Görüntüde görüldüğü üzere make clean, derleme ve ardından arşivleme işlemleri sorunsuz gerçekleşmiştir.

Test 2: Dosya İzinleri ve Arşivden Çıkartma

```
resmi@DESKTOP-4RDP08:/mnt/c/Users/resmi/Desktop/sistem programlama/Sistem Proje Ödevi$ chmod 755 t1.txt
chmod 600 t2.txt
./tarsau -b t1.txt t2.txt -o izinli.sau
./tarsau -a izinli.sau yeni_klasor
ls -la yeni_klasor/
chmod: Operation not permitted (os error 1)
chmod: Operation not permitted (os error 1)
Dosyalar birleştirildi.
yeni_klasor dizininde dosyalar acildi.
total 0
drwxrwxrwx 1 root root 4096 May 24 16:24 .
drwxrwxrwx 1 root root 4096 May 24 18:32 ..
-rwxrwxrwx 1 root root 29 May 24 18:32 t1.txt
-rwxrwxrwx 1 root root 20 May 24 18:32 t2.txt
-rwxrwxrwx 1 root root 21 May 24 17:57 t3.txt
```

```
$ ./tarsau -a sl.sau test_dizini
test_dizini dizininde dosyalar açıldı.
$ ls test_dizini/
t1.txt t2.txt t3.txt

$ cat test_dizini/t1.txt
Bu birinci dosyanın içeriği.

$ cat test_dizini/t2.txt
Bu da ikinci dosya.
$ cat test_dizini/t3.txt
Üçüncü dosya burada.
```

Doğrulama:

- Her dosya orijinal içeriği ve boyutuyla çıkartıldı. **[DOĞRU]**
- Görselde görüldüğü gibi, chmod 755 ve chmod 600 ile belirlenen dosya izinleri arşivden çıkarıldığında orijinal değerleriyle aynen korunmuştur. **[DOĞRU]**

Test 3: Varsayılan arşiv adı (a.sau)

```
$ ./tarsau -b test1.txt test2.txt
Dosyalar birleştirildi.
$ ls a.sau
a.sau
```

Doğrulama: -o parametresi verilmediğinde varsayılan "a.sau" dosya adı kullanıldı. **[DOĞRU]**

Test 4: ASCII Olmayan Dosya Reddi ve Bozuk Arşiv Kontrolü

```
resmi@DESKTOP-4RDPM08:/mnt/c/Users/resmi/Desktop/sistrem programlama/Sistem Proje Ödevi$ ./tarsau -b hatali.txt -o hata.sau
./tarsau -a olmayan_dosya.sau
./tarsau -b
hatali.txt giris dosyasinin formati uyumsuzdur!
Arşiv dosyasi uygunsuz veya bozuk!
Kullanım: tarsau -b dosya1 dosya2 ... [-o cikti.sau] VEYA tarsau -a arşiv.sau [dizin]
```

```
$ cat hatali.txt
Bu dosyada Türkçe karakterler var: ç, ş, ğ, ü, ö, ı

$ ./tarsau -b hatali.txt -o cikti.sau
hatali.txt giriş dosyasının formatı uyumsuzdur!
```

Doğrulama:

- Türkçe/özel karakter içeren veya ASCII olmayan hatali.txt dosyası, format uyumsuzluğu nedeniyle program tarafından yakalanmış ve reddedilmiştir. **[DOĞRU]**
- Ayrıca olmayan veya bozuk dosyalar için de doğru hata mesajları üretilmiştir. **[DOĞRU]**

6. SONUÇ

Projede istenen tüm özellikler başarıyla gerçekleştirilmiştir:

- Metin dosyalarını .sau formatında arşivleme (-b)
- .sau arşiv dosyasından çıkartma (-a)
- Varsayılan arşiv adı (a.sau) desteği
- -o parametresi ile özel arşiv adı belirleme
- ASCII (metin) dosya kontrolü
- Giriş dosyası sayısı sınırı (max 32)
- Toplam boyut sınırı (max 200 MB)
- Hata durumları için uygun mesajlar ve sorunsuz çıkış
- Dosya izinlerinin korunması (chmod ile geri yükleme)
- Dizin oluşturma desteği (mkdir)
- Makefile ile derleme desteği

Program, Linux ortamında C dilinde geliştirilmiş olup POSIX sistem çağrıları (stat, chmod, mkdir, fseek) kullanılmaktadır. Tüm çıkışlar sorunsuz gerçekleşmekte, program aniden çökmemektedir.